

2.3 The Learning Rate

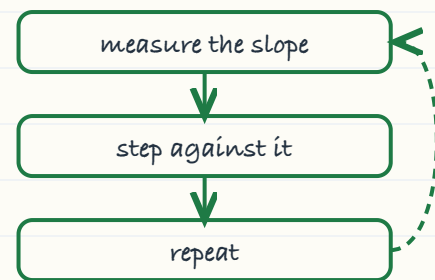
Study Notes · Third Eye Labs
Season 2, episode 3

THE ONE SENTENCE

The gradient says **WHICH WAY**.

The learning rate says **HOW FAR**.

That second number decides whether the model learns, barely learns, or destroys itself.



the loop — this is all training is

1. The update, on one real weight

$$W1[7,11] = -0.2089$$

$$\text{gradient} = -0.0057$$

$$\text{lr} = 0.5$$

$$-0.2089 - 0.5 \times (-0.0057) =$$

$$-0.2061$$

2,410 of these, every single step.

General update (every weight)

$$\theta \leftarrow \theta - \eta \nabla L$$

θ = the weight

η = the learning rate (**HOW FAR**)

∇L = the arrow (**WHICH WAY**)

2. The experiment — one variable (only η changes)

Real network

64 → 32 → 10 (2,410 weights)

1,400 real handwritten digits

300 steps, same seed.

Held-out digits only — testing on what it memorised tells you nothing.

η	loss (start → end)	accuracy	verdict
0.5	2.3162 → 0.0718	12.6% → 97.2%	learns
0.002	2.3162 → 2.1622	12.6% → 28.2%	crawls — still useless
60	pinned at ceiling	12.6% → 8.3%	worse than guessing (10%)

⚠ "A smaller learning rate is safer" is the myth. It is how you wait all day and arrive nowhere.

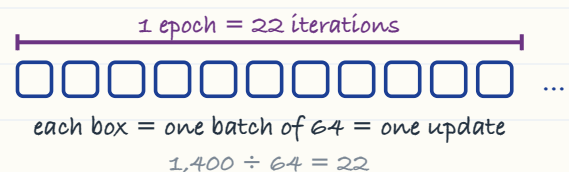
3. Watch it actually learn

9

called 3 → 7 → 3 → 9 → 9

a real 9.99% confident

4. Epoch vs iteration vs batch



1

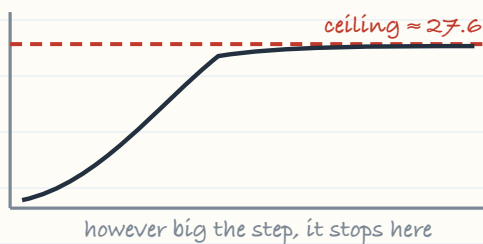
called $7 \rightarrow 2 \rightarrow 1 \rightarrow 1 \rightarrow 1$
 a real 1 · 60% confident

it learned. It did not become infallible — and knowing the difference is most of what using these models well means.

5. Your loss will never reach NaN

...if it is a classifier

lr	peak loss	NAN?
60	25.90	No
2,000	27.63	No
200,000	27.63	No



Where it really breaks: a loss with no ceiling (squared error). Each step multiplies the error by $\sim 29\times$:

$0.0218 \rightarrow 0.204 \rightarrow 5.63 \rightarrow 165 \rightarrow 4,856 \rightarrow$
 $142,700 \rightarrow \dots \rightarrow \text{inf at step 209}$

△ It becomes inf, not NaN. Real NaN comes from arithmetic on infinities ($\text{inf} - \text{inf}$, $0 \times \text{inf}$, $\log 0$).

7. So: $\text{lr} = 3\text{e-}4$

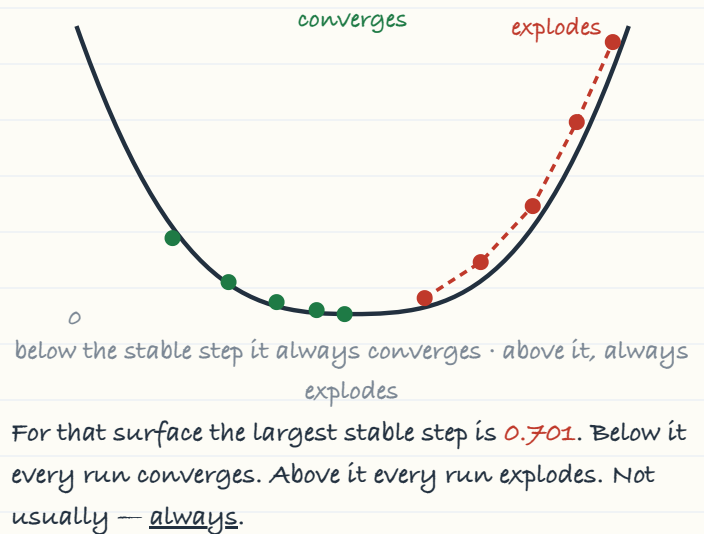
```
optimizer = Adam(model.parameters(),  
lr=3e-4)
```

Three ten-thousandths of the arrow, every step. Small enough not to explode, big enough to actually move — and nobody derived it. Somebody ran the experiment above on a bigger model and that number worked.

An epoch is NOT a unit of time. It is one pass over the data.

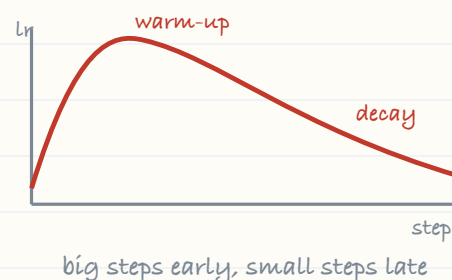
- 2.2 asked how WRONG a small batch makes your arrow (direction).
- 2.3 asks how MANY steps it buys you (counting).

6. The boundary is exact



It comes from the shape of the surface itself, which is why nobody can hand you one learning rate that works for every model.

8. Advanced — tools for the real world



- Schedules: change η over time. Warm-up $\rightarrow$ decay (step / cosine / exponential). Helps stability and final accuracy.
- Adaptive optimisers (Adam, RMSProp): a per-parameter step size, from past gradients. $\rightarrow 2.7$
- Batch size interacts: bigger batch $\rightarrow$ less noise $\rightarrow$ you can usually afford a larger η .
- Diagnose it: watch loss, accuracy, gradient norm, update norm. Loss spikes or accuracy dropping $\rightarrow \eta$

too large. Nothing moving → too small.

NEXT: get the step size perfect and the front of a deep network can still learn nothing → 2.4, vanishing gradients.